# Table of Contents

**Introduction** ..... **2**

**DB Seeding** ..... **3**

**Verifying proxies** ..... **8**

# Demo: Shop Database, Db seeding

Author: Baifan Zhou
Some explanations are created with the help of ChatGPT.

## Introduction

This demo covers two key aspects of managing a shop database: **DB Seeding** and **Handling JSON Output to Avoid Infinite Loops**.
**DB Seeding**:

- **Purpose**: Initialise the database with test data to facilitate development and testing.
- **Implementation**:
  - Create a new class named `DBInit` under the `Models` directory.
  - The `DBInit.cs` code is designed to delete the existing database and recreate it with predefined seed data. If you want to retain existing data, you can comment out specific lines.
  - Seed data includes lists for `Items`, `Customers`, `Orders`, and `OrderItems`.
  - Entity Framework Core automatically handles associations between entities based on foreign key relationships, so explicit navigation properties are not required.

**Verifying proxies:**

- In debugging, we verify that proxies are loaded for navigation properties instead of actual data.

Happy coding!

## DB Seeding

It is common to initialise the database with some seed data for testing. Add a new class named “DBInit.cs” under Models

Change the code of DBInit.cs to below.
Note: Since we now create DB seeding, which is typically for testing the web application. We no longer need the migrations. You can back up the database, because the code (Line11) **always deletes** the existing database and create from scratch (Line12).




Where the four lists of initialisations are as below (complete code in the github repo):

Items:

```
var items = new List<Item>
{
    new Item
    {
        Name = "Pizza",
        Price = 150,
        Description = "Delicious Italian dish with a thin crust topped with tomato sauce, cheese, and
          various toppings.",
        ImageUrl = "/images/pizza.jpg"
    },
    new Item
    {
        Name = "Fried Chicken Leg",
        Price = 20,
        Description = "Crispy and succulent chicken leg that is deep-fried to perfection, often serve
          as a popular fast food item.",
        ImageUrl = "/images/chickenleg.jpg"
    },
    new Item
    {
        Name = "French Fries",
        Price = 50,
        Description = "Crispy, golden-brown potato slices seasoned with salt and often served as a
          popular side dish or snack.",
        ImageUrl = "/images/frenchfries.jpg"
    },
    new Item
    {
        Name = "Grilled Ribs",
        Price = 250,
        Description = "Tender and flavorful ribs grilled to perfection, usually served with barbecue
          sauce.",
        ImageUrl = "/images/ribs.jpg"
    },
    new Item
    {
        Name = "Grilled Ribs",
        Price = 250,
        Description = "Tender and flavorful ribs grilled to perfection, usually served with barbecue
          sauce.",
        ImageUrl = "/images/ribs.jpg"
    },
    new Item
    {
        Name = "Tacos",
        Price = 150,
        Description = "Tortillas filled with various ingredients such as seasoned meat, vegetables,
          and salsa, folded into a delicious handheld meal.",
        ImageUrl = "/images/tacos.jpg"
    },
    new Item
    {
        Name = "Fish and Chips",
        Price = 180,
        Description = "Classic British dish featuring battered and deep-fried fish served with thick-
          cut fried potatoes.",
        ImageUrl = "/images/fishandchips.jpg"
    },
    new Item
    {
        Name = "Cider",
        Price = 50,
        Description = "Refreshing alcoholic beverage made from fermented apple juice, available in
          various flavors.",
        ImageUrl = "/images/cider.jpg"
    },
    new Item
    {
        Name = "Coke",
        Price = 30,
        Description = "Popular carbonated soft drink known for its sweet and refreshing taste.",
        ImageUrl = "/images/coke.jpg"
    },
};
```

Customers:

```
79          if (!context.Customers.Any())
80          {
81              var customers = new List<Customer>
82              {
83                  new Customer { Name = "Alice Hansen", Address = "Osloveien 1"},
84                  new Customer { Name = "Bob Johansen", Address = "Oslomet gata 2"},
85              };
86              context.AddRange(customers);
87              context.SaveChanges();
88          }
```

Orders:

```
90          if (!context.Orders.Any())
91          {
92              var orders = new List<Order>...;
97              context.AddRange(orders);
98              context.SaveChanges();
99          }
```

OrderItems:

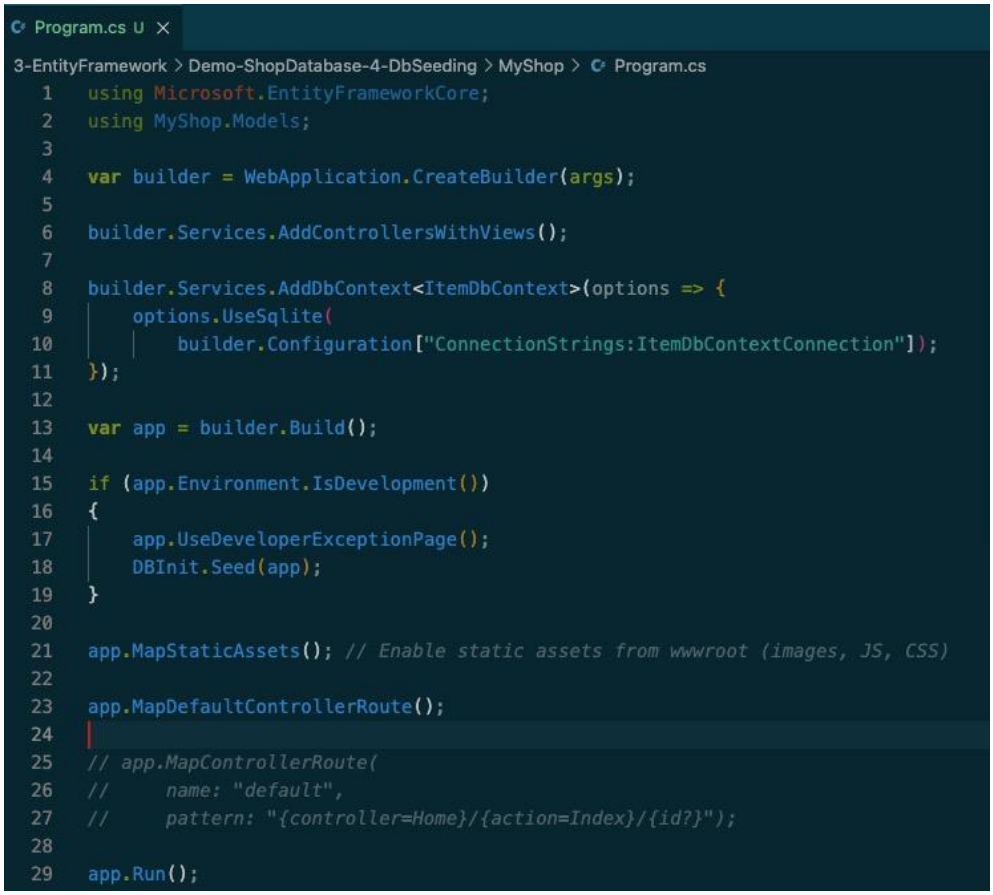
```
101         if (!context.OrderItems.Any())
102         {
103             var orderItems = new List<OrderItem>
104             {
105                 new OrderItem { ItemId = 1, Quantity = 2, OrderId = 1},
106                 new OrderItem { ItemId = 2, Quantity = 1, OrderId = 1},
107                 new OrderItem { ItemId = 3, Quantity = 4, OrderId = 2},
108             };
109             foreach (var orderItem in orderItems)
110             {
111                 var item = context.Items.Find(orderItem.ItemId);
112                 orderItem.OrderItemPrice = orderItem.Quantity * item?.Price ?? 0;
113             }
114             context.AddRange(orderItems);
115             context.SaveChanges();
116         }
```

Update Order.Price:

```
118         var ordersToUpdate = context.Orders.Include(o => o.OrderItems);
119         foreach (var order in ordersToUpdate)
120         {
121             order.TotalPrice = order.OrderItems?.Sum(oi => oi.OrderItemPrice) ?? 0;
122         }
123         context.SaveChanges();
```

Note: that we do not need to add the navigation properties explicitly, because Entity Framework Core handles the associations between entities based on the foreign key properties (e.g., CustomerId in the Order class) and the corresponding primary key properties (e.g., CustomerId in the Customer class). When you create new entities and set their foreign key properties correctly, Entity Framework Core will automatically establish the navigation properties for you.

To initialise the seeding, add Line18, `DBInit.Seed(app);`, which calls Calls the seeding method to initialise the database with predefined data.




**Why We Use This Approach in Development?**
In Program.cs, DBInit.Seed(app) is deliberately placed inside if (app.Environment.IsDevelopment()).

- **Self-Healing Lab Environment:** Calling EnsureDeleted() followed by EnsureCreated() on app startup ensures that every time you run the project ($F5$), your database resets to a known, working state. If you accidentally corrupt, misconfigure, or delete data while building your controllers and views, simply restart the app to fix it.
- **Low Overhead:** You can focus entirely on C#, controllers, views, and LINQ without managing EF Core migration commands (add-migration, update-database).

**Drawbacks of This Approach**
While convenient for lab exercises, this setup has major limitations:

- **No Data Persistence in Dev:** Any new items, customers, or orders you manually create through the browser UI will be wiped out as soon as you restart the application server.
- **Incompatible with Migrations:** EnsureCreated() bypasses EF Core's migration history. If you attempt to run dotnet ef database update on a database created this way, Entity Framework will throw runtime exceptions.
- **High Risk of Data Loss:** Running EnsureDeleted() in a non-testing environment will permanently destroy all existing database content.

**How is the launch mode decided for Dev or Production?**

When you execute dotnet run, .NET reads the **Properties/launchSettings.json** file in your project directory. This file defines startup profiles that set the **ASPNETCORE_ENVIRONMENT** environment variable to "Development". When dotnet run executes, it picks the default profile from this file and injects ASPNETCORE_ENVIRONMENT=Development into the process memory.

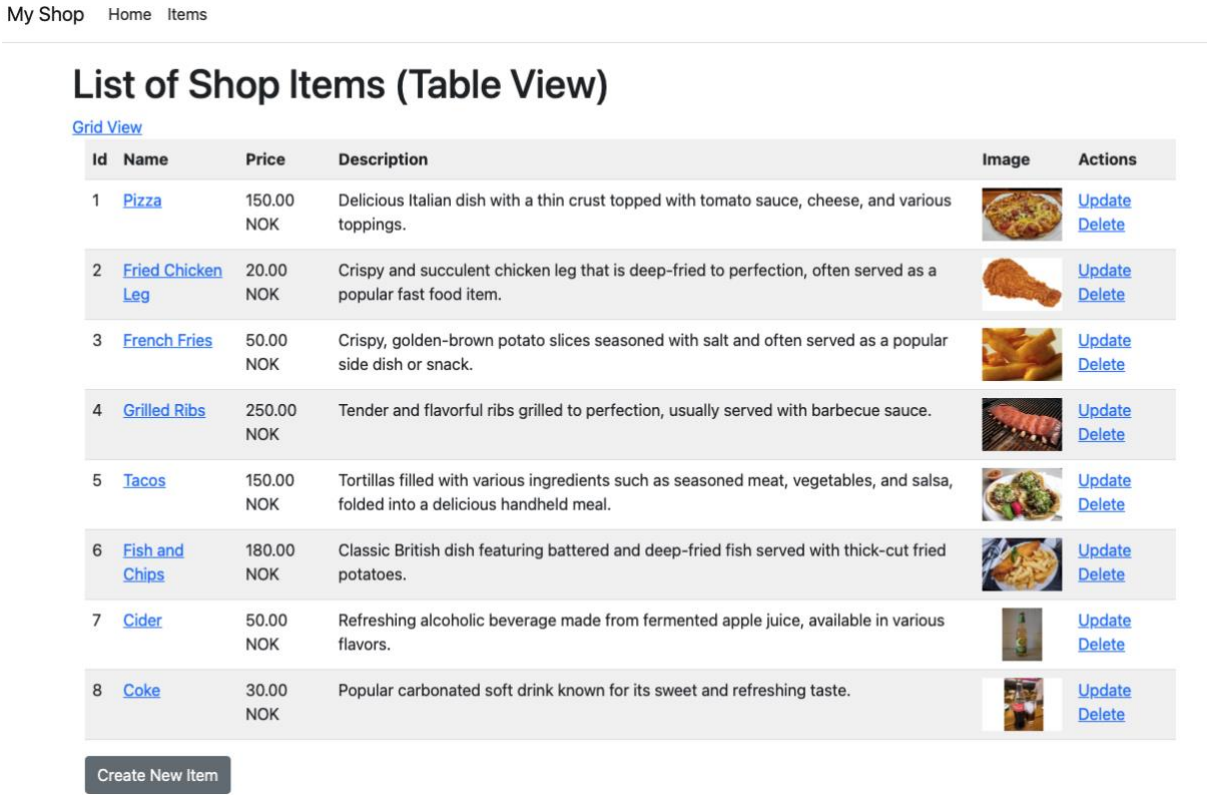
**What is different for Production**

Because Program.cs guards DBInit.Seed(app) inside app.Environment.IsDevelopment(), **your production database is safe from being deleted.** However, to make the application fully production-ready (or for exam requirements that assess database persistence):

1. **Migrations Are Required for Production:** EnsureCreated() bypasses EF Core's migration tracking. To build a database schema in production, you must generate migration files using dotnet ef migrations add InitialCreate.
2. **Production Schema Deployment:** In production, schema changes are applied safely using context.Database.Migrate() or deployment scripts, never EnsureCreated().
3. **Persistent Testing (If required in your project):** If your exam project design requires to preserve browser-submitted data across app restarts, remove EnsureDeleted() and EnsureCreated() from DBInit.cs, run context.Database.Migrate(), and let the if (!context.Items.Any()) check handle non-destructive seeding.

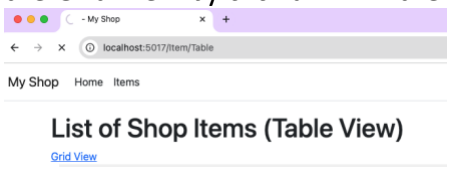
For the demo, let's keep this simple approach.

Run the project with dotnet run or using debugging (F5), go to Items. You can see the list of seeding data is created

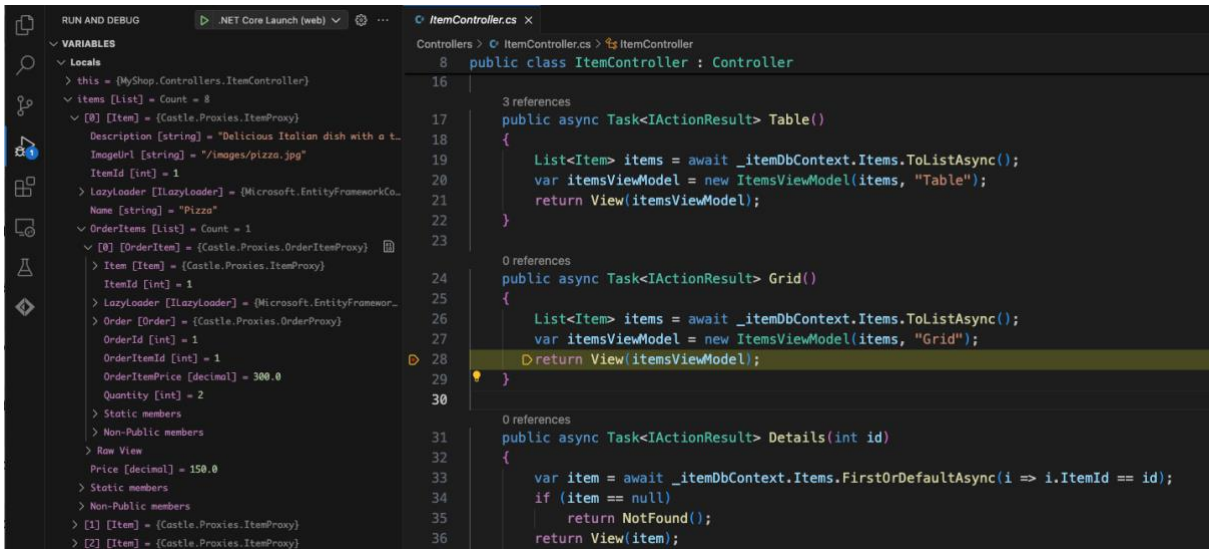


## Verifying proxies

During the debugging, make a breakpoint, e.g., Line 28 in ItemController.cs, and then go to the Grid View by click a link in the Table View




In the debugging tab, you will see that all navigation properties are replaced with proxies, not their actual data.




Now you have a web app with a lot of content! Great work! ☺